Packages in Java – Command-Line Practice Sheet-7	
Aim:	To understand and practice creating a Java package in one class, importing that package in another class located in a separate directory, and compiling/running the programs from the command line using javac, java, -cp, -d, . and .. paths.
Tools/Environment:	JDK 17 or later / Command Prompt or Terminal / VS Code (for editing)
Prerequisites:	Basic Java syntax, classes and objects, methods, import statement, javac and java commands, and basic command-line directory navigation
Practice Sheet – Theory and Solved Questions	

Part A – Theory

1. What is a Package in Java

A package is a namespace used to organize related Java classes and interfaces. A package also helps avoid naming conflicts and supports access control. A class declares its package with a package statement, normally as the first non-comment statement in the source file.

2. Package Declaration

The package statement associates a class with a package. For example, package college.tools; means the class belongs to the college.tools package. The package name and directory structure should correspond when classes are compiled and stored in the normal package layout.

3. Directory Structure and Package Structure

For package college.tools, a conventional source location is src/college/tools/Calculator.java. The source root is src; college/tools represents the package path. The package name is not the same as the complete file path: it is mapped to directories below the source or classpath root.

4. Import Statement

The import statement allows another class to be referred to by its simple name instead of its fully qualified name. For example, import college.tools.Calculator; allows code in another package to use Calculator when the compiled class is available on the classpath.

5. Fully Qualified Class Name

A fully qualified class name contains the package name and class name, such as college.tools.Calculator. It can be used without an import statement, for example college.tools.Calculator c = new college.tools.Calculator();.

6. Separate Directories for Two Classes

In this lab, the package class and the class that imports it are intentionally stored in separate directories. Example: src/college/tools/Calculator.java and src/app/Main.java. Main imports college.tools.Calculator.

7. Source Root and Classpath Root

A source root is the directory from which package directories begin when source files are organized. A classpath root is the directory from which Java searches for package directories containing compiled classes. If out is the classpath root, Java looks for college.tools.Calculator as out/college/tools/Calculator.class.

8. javac Command

javac is the Java compiler. It converts .java source files into .class bytecode files. For example, javac Main.java compiles Main.java. When packages are involved, the output directory should normally be controlled with -d.

9. java Command

java launches a compiled Java class. With packages, the class is normally specified by its fully qualified name, such as java -cp out app.Main. Do not add .class to the class name.

10. -d Option

The javac -d option specifies the destination directory for compiled class files. For example, javac -d out source.java places generated class files under out while creating the required package-directory structure.

11. -cp / -classpath Option

The -cp or -classpath option tells javac or java where to find classes and other classpath resources. For example, java -cp out app.Main tells the JVM that out is a classpath root.

12. Compiling the Package Class First

One useful learning workflow is to compile the package class first into an output directory and then compile the importing class with that output directory on its classpath. This clearly demonstrates why -cp is required when the dependency is already compiled.

13. Running with -cp

If compiled classes are stored under out, use java -cp out app.Main from the project root. The JVM searches out for app/Main.class and, when required, for college/tools/Calculator.class.

14. Single Dot (.) in Paths

A single dot represents the current working directory. Therefore -cp . means use the current directory as a classpath root. If app/Main.class is located under the current directory, java -cp . app.Main can run it.

15. Double Dot (..) in Paths

Two dots represent the parent directory of the current working directory. For example, if the current directory is project/run, .. refers to project. A path such as ../out refers to the out directory inside the parent directory. The meaning of .. depends on the current working directory.

16. Relative Path

A relative path is interpreted from the current working directory. Examples include ., .., ./out, ../out and src/college/tools. Relative paths are convenient but depend on where the command is executed.

17. Absolute Path

An absolute path starts from the filesystem root and does not depend on the current directory. Windows examples can use D:\JavaPackageDemo\out, while Linux examples can use /home/student/JavaPackageDemo/out. An absolute classpath can therefore be used even when the terminal is in another directory.

18. Windows and Linux Classpath Separators

When multiple classpath entries are supplied, Windows uses a semicolon (;), while Linux and macOS normally use a colon (:). A single classpath entry such as -cp out does not require a separator.

19. Important Compilation Rule

The package declaration, directory layout, source root, classpath root and fully qualified class name must be understood together. A common mistake is to place the package directory itself on the classpath instead of its parent classpath root. For package college.tools, the classpath root should contain college/tools.

20. Recommended Lab Project Structure

Use a separate source directory and output directory so that source files and compiled files are easy to identify. The example used throughout this practice sheet is: JavaPackageDemo/src/college/tools/Calculator.java, JavaPackageDemo/src/app/Main.java and JavaPackageDemo/out/college/tools/Calculator.class plus out/app/Main.class.

Reference Project Structure Used in This Practice Sheet

```
JavaPackageDemo/
├── src/
│   ├── college/
│   │   └── tools/
│   │       └── Calculator.java
│   └── app/
│       └── Main.java
└── out/
    ├── college/
    │   └── tools/
    │       └── Calculator.class
    └── app/
```

```
        └── Main.class
```

Calculator.java belongs to package college.tools. Main.java is in package app and imports college.tools.Calculator. The source files are in separate directories, and the compiled classes are also stored in their corresponding package directories under out.

Part B – 20 Solved Practice Questions

1. Create the package class Calculator.java in one directory and declare it as package college.tools.

Solution:

```
// File: src/college/tools/Calculator.java
package college.tools;

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

The package declaration makes Calculator a member of college.tools. The source file is placed under the matching package directory.

2. Create a second class Main.java in a separate directory and import Calculator from the package.

Solution:

```
// File: src/app/Main.java
package app;

import college.tools.Calculator;

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Sum = " + c.add(10, 20));
    }
}
```

Main is in the app package and imports college.tools.Calculator. The two source files are physically stored in separate directories.

3. Create the complete directory structure from the command line on Windows.

Solution:

```
mkdir JavaPackageDemo
cd JavaPackageDemo
mkdir src
mkdir src\college
mkdir src\college\tools
mkdir src\app
mkdir out
```

The commands create separate source directories for college.tools and app, plus the output directory. On Linux, use / instead of \ in paths.

4. Create the complete directory structure from the command line on Linux.

Solution:

```
mkdir -p JavaPackageDemo/src/college/tools
mkdir -p JavaPackageDemo/src/app
mkdir -p JavaPackageDemo/out
cd JavaPackageDemo
```

mkdir -p creates the required parent directories automatically. The project root is now the current working directory.

5. Compile both source files at once using javac and -d out.

Solution:

```
javac -d out src/college/tools/Calculator.java src/app/Main.java
```

The -d out option sends compiled classes to out. javac creates out/college/tools/Calculator.class and out/app/Main.class according to their package names.

6. Display the compiled directory structure and verify that -d created package directories.

Solution:

```
Windows:
tree out

Linux:
find out -type f
```

The output should show Calculator.class below out/college/tools and Main.class below out/app. This verifies that -d preserved the package structure.

7. Compile the package class first, then compile Main.java using -cp out.

Solution:

```
javac -d out src/college/tools/Calculator.java
javac -cp out -d out src/app/Main.java
```

The first command creates Calculator.class under out. The second command uses -cp out so javac can find the already compiled college.tools.Calculator class.

8. Run the program from the project root using -cp out.

Solution:

```
java -cp out app.Main
```

out is the classpath root, and app.Main is the fully qualified name of the class containing main(). Do not write app.Main.class.

9. Run the same program using the single-dot classpath.

Solution:

```
cd out
java -cp . app.Main
```

Here . means the current directory, which is out. Because out contains app/Main.class and college/tools/Calculator.class, the program can run from this directory.

10. Explain and demonstrate why java -cp . app.Main works from the out directory but not necessarily from the project root.

Solution:

```
From project root:
java -cp . app.Main

From project root, the command normally fails because . contains src and out, not
app/Main.class directly.

From out:
cd out
java -cp . app.Main
```

The dot always means the current working directory. It does not mean the project root automatically. The classpath root must be the directory that contains the package directory app.

11. Demonstrate the double-dot path by running from a child directory.

Solution:

```
cd out
mkdir run
cd run
java -cp .. app.Main
```

Inside out/run, .. refers to out. Therefore -cp .. makes out the classpath root, where app/Main.class and college/tools/Calculator.class exist.

12. Demonstrate a relative path using ../out from a directory below the project root.

Solution:

```
cd run
java -cp ../../out app.Main
```

If run is JavaPackageDemo/run, ../../out points to JavaPackageDemo/out. The exact number of .. components depends on the current directory.

13. Demonstrate an absolute Windows classpath.

Solution:

```
java -cp D:\JavaPackageDemo\out app.Main
```

D:\JavaPackageDemo\out is an absolute path. It identifies the classpath root independently of the current directory.

14. Demonstrate an absolute Linux classpath.

Solution:

```
java -cp /home/student/JavaPackageDemo/out app.Main
```

The Linux path starts at /, so it is absolute. Replace /home/student/JavaPackageDemo with the actual project location.

15. Compile Main.java with an absolute classpath and an absolute output directory.

Solution:

```
Windows example:
javac -cp D:\JavaPackageDemo\out -d D:\JavaPackageDemo\out
D:\JavaPackageDemo\src\app\Main.java

Linux example:
javac -cp /home/student/JavaPackageDemo/out -d /home/student/JavaPackageDemo/out
/home/student/JavaPackageDemo/src/app/Main.java
```

Both -cp and -d accept paths. Here -cp points to already compiled dependencies and -d specifies where the new class file is written.

16. Compile and run using only relative paths from the project root.

Solution:

```
javac -d out src/college/tools/Calculator.java
javac -cp out -d out src/app/Main.java
java -cp out app.Main
```

These commands are relative to the project root. This is usually easier to teach and maintain because the commands do not depend on an absolute machine-specific location.

17. Use the fully qualified class name without an import statement.

Solution:

```
// File: src/app/Main.java
package app;

public class Main {
    public static void main(String[] args) {
        college.tools.Calculator c = new college.tools.Calculator();
        System.out.println("Sum = " + c.add(30, 40));
    }
}

Compile and run:
javac -d out src/college/tools/Calculator.java src/app/Main.java
java -cp out app.Main
```

The import statement is optional when the fully qualified class name is used directly. The classpath requirement remains the same.

18. Demonstrate a common classpath mistake and correct it.

Solution:

```
Wrong from the project root:
java -cp out/college/tools app.Main

Correct:
java -cp out app.Main
```

The classpath must point to the directory that contains the package root college, not to college/tools itself. Java then maps app.Main to out/app/Main.class.

19. Perform a complete clean compile and run using separate source and output directories.

Solution:

```
Windows:
rmdir /s /q out
mkdir out
javac -d out src\college\tools\Calculator.java
javac -cp out -d out src\app\Main.java
java -cp out app.Main

Linux:
rm -rf out
mkdir out
javac -d out src/college/tools/Calculator.java
javac -cp out -d out src/app/Main.java
java -cp out app.Main
```

This workflow removes old class files, recompiles the package class, compiles the importing class using -cp, and finally runs the fully qualified main class.

20. Complete package workflow: create two classes in separate directories, compile with -d and -cp, and run using ., .. and an absolute path.

Solution:

```
Project structure:
JavaPackageDemo/
├── src/college/tools/Calculator.java
├── src/app/Main.java
└── out/

Compile from project root:
javac -d out src/college/tools/Calculator.java
javac -cp out -d out src/app/Main.java

Run from project root:
java -cp out app.Main

Run from out:
cd out
java -cp . app.Main

Run from out/run:
mkdir run
cd run
java -cp .. app.Main

Run with an absolute path (replace with your real path):
java -cp D:\JavaPackageDemo\out app.Main

Linux absolute-path form:
java -cp /home/student/JavaPackageDemo/out app.Main
```

This final workflow demonstrates package declaration, separate directories, import, -d, -cp, the current-directory dot, the parent-directory double dot, relative paths and absolute paths.

Quick Reference – . and .. and Path Types

```
.       = current working directory
..      = parent directory of the current working directory
./out   = out directory inside the current directory
../out  = out directory inside the parent directory

Relative path examples:
-cp out
-cp ./out
-cp ../out
```

```
Absolute path examples:
Windows: -cp D:\JavaPackageDemo\out
Linux:   -cp /home/student/JavaPackageDemo/out
```

Important: . and .. are path components interpreted by the operating system shell/filesystem. They do not have a fixed location. Their meaning changes when you change the current working directory.

Part C – Revision and Command-Line Checklist

- Package name and directory structure must match the intended package hierarchy.
- Keep the package class and importing class in separate directories for this lab.
- Use javac to compile .java files and java to run the compiled class.
- Use -d to choose the destination directory for compiled .class files.
- Use -cp or -classpath to tell javac/java where classes can be found.
- The classpath root is the directory above the package directory, not the package directory itself.
- A single dot (.) means the current working directory.
- A double dot (..) means the parent directory of the current working directory.
- Relative paths depend on the current directory; absolute paths identify a location independently of it.
- For a packaged class, run it with its fully qualified name, such as app.Main.
- Do not write .class after the class name when using the java command.
- Windows uses ; between multiple classpath entries; Linux/macOS normally use :.
- After changing directories, re-check what . and .. refer to before using a relative classpath.
- Verify the output directory after javac -d by checking the generated package folders and .class files.